



Image generated by ChatGPT.

# Sports Manager

Assoc. Prof. Dr. Kaya Oğuz

| Version | Date       | Comments         |
|---------|------------|------------------|
| 1       | 01.03.2026 | Initial version. |

## Introduction

Football is one of the most interesting games in Turkey and the world. The games themselves attract large crowds and fans support their teams with fanatic devotion. That's why games like Football Manager have become a yearly attraction, with new engines, new rosters, and new features waiting for the players.

Football Manager used to be called Championship Manager, and back in the 1990s, it did not have a graphical engine to simulate the



## Sports Manager

Following the same tradition, your project is to develop a GUI based manager game which will implement only a subset of the features of these games. The interesting part is that it is not specific to football, and with your knowledge of object oriented programming, you should be able to add other sports, such as volleyball, as the player starts a new game. This will change several things in the game, such as player attributes, but it will still have some consistent structure so that your application can handle the UI.

### Headball Example

Let me explain the game with an unreal sport called "headball" - you can replace it with any other sports such as football, basketball, volleyball, etc.

When a new game starts, the game will ask what sport we want to be a manager of; and we pick headball. This generates a league of, say, 20 headball teams, where each team can have several players, all of them randomly generated with their attributes. In headball, two teams compete with a total of 7 players and 3 substitutes. Each of these players have a position specific to headball, and the team can employ a handball tactic. The team will also have a list of coaches who can train the players. The club finances are ignored.

The generated teams will only compete in the league (no need to add other competitions). In the handball league, each team plays every other team twice. A weekly fixture is created and the player can see the schedule of the team, the league table, and the player list. During the week there will be training, during the weekend there will be the matches.

Before a match, the player picks the team, sets the tactic, and proceeds to the match. A headball game is played in four quarters. During the play, the player cannot control the team, but once a quarter ends, the player can make changes to tactics or substitute players. In headball, you can substitute players as many times as required. Once the game is over, the winner gets 2 points and a draw is 1 point for each team. No points are awarded to the losing team. Headball games score a lot of points, so it is common to have scores in the ranges 10 to 20. Also, at the end of the game, one or more players can be injured for some number of games (not weeks), so when a player is injured, they cannot be in the team for a match.

The league standings for headball are on the points of the teams. If the points are the same, the match between the teams are considered,



## Framework

Since headball is not a real game, you can start the development with football. However, your app should be ready to include one or more sports later in the development. Therefore, all the sports related functionality must be open to extension.

This requires a framework and a clear separation of the details from other parts of the software, specifically the UI. While there will be teams, players, coaches, training, matches, leagues, etc. their contents, rules, and outcomes will change significantly. Your UI should rely on an interface (in the sense of Java interface), rather than a real implementation. So, for your first milestone, your job is to make abstractions so that they can later be replaced. Some time during the semester, I will announce the other sport, and your team must also implement that sport, too.

## User Interface and Data

Keep the user interface modern and clean. Use AI to generate male and female names for players and coaches, several logos that can be assigned to teams for visuals, team names that can be loaded to the application and can be used to generate them when a new game begins.

Take a look at BygFoot (<https://bygfoot.sourceforge.io/new/index.html>) but make sure you get a better UI.

## Milestones

There are going to be three milestones.

**Milestone 1:** You will deliver a document for the design of the software. Please notice that the app relies heavily on abstraction. There is going to be a “sport” but the app does not know it, the details will be implemented in concrete sport classes. Pay attention to this step. It is possible to change the design during implementation, so don’t worry about that. However, take note, and discuss it in the final project report.

**Milestone 2:** You will deliver the foundation classes and unit tests. These include the interfaces, abstract classes, concrete classes, and the tests that demonstrate they work as intended. The tests should cover as much as possible.

**Milestone 3:** This is the project delivery. As discussed in the class, your app must have an



## Project Document

Updated automatically every 5 minutes

reasons for your design choices, which object-oriented principles you have applied and where you have applied them. You should also include a “post-mortem” which discusses what went right and what went wrong.

## Rubric

|                                            | LO  | Low (0 - 44)                                                                                                                                                                                             | Mid (45 - 79)                                                                                                                                                                                                                                     | High (80 - 100)                                                                                                                                                                                                                                                                                                | Total |
|--------------------------------------------|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| M1: Design and Architecture Document (10%) | LO1 | The document is missing, incomplete, or fails to address the core requirement of abstraction/extensibility for multiple sports. The design is poorly structured or presented.                            | The document is present and attempts to use abstraction but the design is inconsistent, incomplete, or only partially covers the required separation of concerns, such as UI from sport logic,                                                    | A comprehensive design document with a well-defined architecture that fully embraces abstraction (using Java interfaces/abstract classes) for easy inclusion of new sports. Clear separation of sport-specific logic from the UI. Excellent structure, clarity, and professional presentation.                 |       |
| M2: Foundation Classes and Tests (30%)     | LO5 | Foundation classes are incomplete, do not implement the required abstractions, or contain significant errors. Unit tests are missing, minimal, or fail to demonstrate that the core functionality works. | Foundation classes are mostly complete and follow the design, but required abstraction principles are missed or inconsistently applied. Unit tests cover basic functionality but lack comprehensive coverage for various scenarios or edge cases. | All required interfaces, abstract, and concrete classes are well-defined, robust, and correctly implement the required object-oriented abstraction principle. Comprehensive and effective unit tests with high coverage that robustly demonstrate the intended functionality of the core framework components. |       |

Project Document

Updated automatically every 5 minutes

|                                               |     |                                                                                                                                                                                   |                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                             |  |
|-----------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
|                                               |     |                                                                                                                                                                                   | functional or highly unstable.                                                                                                                                                                                           | may be brief or unclear. The final report covers the required topics but lacks depth in discussing design changes, applied OO principles, or the post-mortem. A basic, functional application.                                                                              | and a comprehensive, clear user manual. A detailed final report that thoroughly discusses design changes and justifications, analyzes the application of object-oriented principles, and includes a thoughtful post-mortem. |  |
| Team Contribution, (Your TPS and Git Records) | LO7 | Inconsistent or minimal Git activity, making it difficult to assess individual contribution. Team progress records (TPS) are missing or very poor, indicating poor collaboration. | Adequate Git activity demonstrating regular contribution, but with some periods of inactivity or less effective collaboration. TPS/records are present but may be generic or lack specific details on work distribution. | Consistent, high-quality, and meaningful contributions from all team members, as clearly evidenced by a well-documented Git history (commits, branches, etc.). Excellent, detailed, and accurate team progress records demonstrating effective and equitable collaboration. |                                                                                                                                                                                                                             |  |
| Total                                         |     |                                                                                                                                                                                   |                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                             |  |